

Taller Bintec — Guía Completa

Automatización bancaria multiagente con n8n, Gemini API y Python

1. Requerimientos para el taller Bintec

Entorno local

- n8n instalado o corriendo (local o en contenedor)
- Docker Desktop o Docker Engine
- Python 3.11+
- VS Code o un editor similar
- Postman instalado

Google Cloud

- Cuenta de Google para crear la Gemini API key
- Acceso a Google AI Studio
- Permisos para habilitar APIs en el proyecto de Google Cloud asociado

2. Antes de Empezar — Arquitectura

El flujo conecta un cliente (WhatsApp, un formulario, o un curl de prueba) con un webhook de n8n. n8n actúa como agente orquestador: consulta tres servicios de negocio (Risk, Fraud, CRM, todos Python + FastAPI), envía el contexto combinado a Gemini para que tome una decisión, y esa decisión pasa por un guardrail de reglas de negocio y una capa de auditoría antes de responder — con un humano en el loop para los casos sensibles.

- **Cliente / WhatsApp / Webhook** — origen de la solicitud
- **n8n Workflow · Agente Orquestador (Gemini)** — el cerebro del flujo
- **Risk API** (Python + FastAPI) — score, endeudamiento, riesgo crediticio
- **Fraud API** (Python + FastAPI) — IP, dispositivo, monto, patrones
- **CRM API** (Python + FastAPI) — perfil, ingresos, historial de producto
- **Decisión de Gemini + auditoría + guardrail humano** — salida final

3. Servicios Mock (Python + FastAPI)

Los tres servicios simulan sistemas bancarios reales. Cada uno expone `/health` y un endpoint de negocio, y corren dentro de Docker vía docker-compose.

Risk API — `mock_services/risk_service.py`

```

from fastapi import FastAPI
from pydantic import BaseModel
from fastapi.responses import JSONResponse

app = FastAPI(title="Risk API")

class RiskRequest(BaseModel):
    customer_id: str
    requested_amount: float
    term_months: int

@app.get("/health")
def health():
    return {"status": "ok", "service": "risk-api"}

@app.post("/score")
def score(request: RiskRequest):
    base = 750
    amount_factor = 100 - (request.requested_amount / 150000)
    term_factor = (30 - request.term_months) / 2
    score = int(min(850, max(400, base + amount_factor + term_factor)))
    if score >= 700:
        level = "BAJO"; approved_limit = 25000000
    elif score >= 550:
        level = "MEDIO"; approved_limit = 12000000
    else:
        level = "ALTO"; approved_limit = 4000000
    return {"customer_id": request.customer_id, "credit_score": score, "risk_level":
level,
        "approved_limit": approved_limit, "recommended_term_months":
min(request.term_months, 36)}

if __name__ == "__main__":
    import uvicorn; uvicorn.run(app, host="0.0.0.0", port=8000)

```

Fraud API — mock_services/fraud_service.py

```

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="Fraud API")

class FraudRequest(BaseModel):
    customer_id: str; ip: str; channel: str; amount: float; device_id: str

@app.get("/health")
def health():
    return {"status": "ok", "service": "fraud-api"}

@app.post("/check")
def check(request: FraudRequest):
    suspicious_ip = request.ip.startswith("185.") or request.ip.startswith("10.")
    high_amount = request.amount > 8000000
    risky_channel = request.channel.lower() in ["sms", "email"]
    risk_score = 0
    if suspicious_ip: risk_score += 30
    if high_amount: risk_score += 25
    if risky_channel: risk_score += 20
    return {"customer_id": request.customer_id, "fraud_risk_score": risk_score,
        "is_high_risk": risk_score >= 45, "alerts": {"suspicious_ip": suspicious_ip,
        "high_amount": high_amount, "risky_channel": risky_channel}}

if __name__ == "__main__":
    import uvicorn; uvicorn.run(app, host="0.0.0.0", port=8000)

```

CRM API — mock_services/crm_service.py

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="CRM API")

class CRMRequest(BaseModel):
    customer_id: str

@app.get("/health")
def health():
    return {"status": "ok", "service": "crm-api"}

@app.post("/profile")
def profile(request: CRMRequest):
    customer_map = {
        "CLI-893201": {"customer_name": "Mariana Gómez", "segment": "PREFERENCIAL",
            "monthly_income": 6200000, "employment": "Empleado", "tenure_months": 42,
            "product_history": ["tarjeta", "cuenta de ahorros"]},
        "CLI-440102": {"customer_name": "Andrés Ramírez", "segment": "MEDIO",
            "monthly_income": 3200000, "employment": "Independiente",
            "tenure_months": 18,
            "product_history": ["cuenta corriente"]},
    }
    record = customer_map.get(request.customer_id, {"customer_name": "Cliente nuevo",
        "segment": "NUEVO", "monthly_income": 2500000, "employment": "No reportado",
        "tenure_months": 0, "product_history": []})
    return {"customer_id": request.customer_id, **record}

if __name__ == "__main__":
    import uvicorn; uvicorn.run(app, host="0.0.0.0", port=8000)
```

Los 3 servicios para levantar

4. docker-compose.yml

Levanta n8n y los tres servicios mock con un solo comando.

```
services:
  n8n:
```

```
image: n8nio/n8n:latest
ports: ["5678:5678"]
environment: ["N8N_SECURE_COOKIE=false"]
volumes: ["n8n_data:/home/node/.n8n"]
risk-api:
  image: python:3.12-slim
  working_dir: /app
  volumes: ["/mock_services:/app"]
  command: bash -lc "pip install fastapi uvicorn && python risk_service.py"
  ports: ["8001:8000"]
fraud-api:
  image: python:3.12-slim
  volumes: ["/mock_services:/app"]
  command: bash -lc "pip install fastapi uvicorn && python fraud_service.py"
  ports: ["8002:8000"]
crm-api:
  image: python:3.12-slim
  command: bash -lc "pip install fastapi uvicorn && python crm_service.py"
  ports: ["8003:8000"]
volumes:
  n8n_data:
```

Cómo hacerlo

- Crea la carpeta principal con la subcarpeta `mock_services/` y los tres archivos `.py` de arriba.
- Guarda este `docker-compose.yml` en la raíz del proyecto (junto a `mock_services/`).
- Abre una terminal en la carpeta raíz y ejecuta el comando de abajo.

```
docker compose up -d
```

5. Configurar la API de Gemini

- Acceder a Google AI Studio — con la cuenta de Google que usarás para el taller, entra a aistudio.google.com/api-keys.
- Crear la API key — haz clic en "Create API key" y elige o crea el proyecto de Google Cloud asociado.
- Copiar la API key generada — esa clave es la que vas a pegar como credencial en n8n en el siguiente paso.

Guarda esa API key — la necesitas en el siguiente paso para crear la credencial en n8n.

6. Credenciales en n8n

- En n8n ve a **Personal** → **Credentials**.
- Haz clic en **Crear credencial** y elige el tipo Header Auth.
- Completa los campos de la tabla y guarda la credencial como **x-goog-api-key**.

Campo	Valor
Name	x-goog-api-key
API Key	tu clave de Gemini
Allowed HTTP Requests	all

Esta credencial es la que usará el nodo HTTP Request → Gemini en el flujo de n8n.

7. Flujo en n8n — Nodo por Nodo (15 nodos)

Arma estos 15 nodos, en orden, de Webhook a Webhook Response. Ejecuta el flujo una vez de prueba (sin datos reales) — si los 15 nodos corren sin error, continúa a la trazabilidad.

Paso 1: Webhook

Recibe la solicitud bancaria.

Campo	Valor
Test URL	.../webhook-test/solicitud-bancaria
Method	POST
Path	solicitud-bancaria
Authentication	none
Respond	Using "Respond to Webhook" node

Paso 2: Set — mapear campos

Mapea los campos de entrada.

Campo	Expresión
customer_id	{{ \$json.body.customer_id }}
requested_amount	{{ \$json.body.requested_amount }}
term_months	{{ \$json.body.term_months }}
channel / ip / device_id	{{ \$json.body.channel }}, .ip, .device_id

Paso 3: HTTP Request → Risk API

Consulta el servicio de riesgo crediticio.

Campo	Valor
Method	POST
URL	http://risk-api:8000/score
Authentication	none
Body Content Type	JSON

JSON Body:

```
{{ JSON.stringify({
  customer_id: $json.customer_id,
  requested_amount: $json.requested_amount,
  term_months: $json.term_months
}) }}
```

Paso 4: HTTP Request → Fraud API

Consulta el servicio de detección de fraude.

Campo	Valor
Method	POST
URL	http://fraud-api:8000/check
Authentication	none
Body Content Type	JSON

JSON Body:

```
{{ JSON.stringify({
  customer_id: $json.customer_id,
  ip: $json.ip,
  channel: $json.channel,
  amount: $json.requested_amount,
  device_id: $json.device_id
}) }}
```

Paso 5: HTTP Request → CRM API

Consulta el perfil del cliente.

Campo	Valor
Method	POST

Campo	Valor
URL	http://crm-api:8000/profile
Authentication	none
Body	{{ JSON.stringify({ customer_id: \$json.customer_id }) }}

Paso 6: Merge — combinar respuestas

Conecta las salidas de Risk, Fraud y CRM a las tres entradas del nodo Merge, en ese orden.

Campo	Valor
Mode	Combine
Combine by	Position
Number of inputs	3

Paso 7: Code — reestructurar datos

El Merge combina en un objeto plano. Este nodo lo reorganiza en una estructura anidada: cliente / riesgo / fraude.

```
const flat = $json;
return [{
  json: {
    cliente: { customer_id: flat.customer_id, customer_name: flat.customer_name,
segment: flat.segment,
monthly_income: flat.monthly_income, employment: flat.employment,
tenure_months: flat.tenure_months,
product_history: flat.product_history },
riesgo: { customer_id: flat.customer_id, credit_score: flat.credit_score,
risk_level: flat.risk_level,
approved_limit: flat.approved_limit, recommended_term_months:
flat.recommended_term_months },
fraude: { customer_id: flat.customer_id, fraud_risk_score: flat.fraud_risk_score,
is_high_risk: flat.is_high_risk, alerts: flat.alerts },
}
}];
```

Paso 8: HTTP Request → Gemini (AI Agent)

El modelo decide sobre la solicitud, en base a todo el contexto reunido.

Campo	Valor
Method / URL	POST .../gemini-3.5-flash-lite:generateContent
Authentication	Header Auth → x-goog-api-key
Body Type	JSON

JSON Body completo del nodo:

```
{{ JSON.stringify({
  system_instruction: {
    parts: [{
      text: "Eres un agente bancario experto en riesgo y atención al cliente. Tu
misión es revisar la información financiera y de seguridad de un cliente para decidir
si la solicitud puede ser aprobada, rechazada o requiere revisión manual. Revisa los
datos del cliente, el score de riesgo, la evaluación de fraude y la capacidad de
endeudamiento. El campo decision DEBE ser exactamente uno de estos cuatro valores,
sin variaciones: APROBADO, APROBADO_CON_RESTRICCIONES, RECHAZADO,
PENDIENTE_REVISION_HUMANA. Usa APROBADO solo cuando no exista ninguna alerta de
fraude activa (alerts.suspicious_ip, alerts.high_amount, alerts.risky_channel todas
en false) y el monto aprobado sea igual al límite completo. Usa
APROBADO_CON_RESTRICCIONES cuando el riesgo sea bajo o medio pero exista al menos una
alerta de fraude activa, o cuando el monto aprobado sea menor al solicitado. Usa
RECHAZADO cuando el riesgo sea alto y no proceda ninguna aprobación. Usa
PENDIENTE_REVISION_HUMANA solo si tú mismo consideras que el caso es ambiguo y
requiere que un humano decida. Devuelve un resultado con: decisión, razón, monto
sugerido, riesgo y mensaje final para el cliente. Hazlo con lenguaje claro,
conservador y orientado a la seguridad. Responde ÚNICAMENTE con un objeto JSON válido
(sin texto adicional, sin markdown) con las claves: decision, risk_level,
approved_amount, reason, human_review, customer_message."
    }]
  },
  contents: [
    { parts: [ { text: JSON.stringify($json) } ] }
  ]
}) }}
```

Paso 9: Code — parsear la respuesta de Gemini

Gemini devuelve el JSON envuelto en markdown. Este código lo extrae y lo parsea.

```
// Gemini devuelve { candidates: [{ content: { parts: [{ text: '...' }] } } ] }
// Extrae el texto y lo parsea
let raw = $json.candidates[0].content.parts[0].text;
// Si viene en bloque markdown, lo limpia
if (raw.startsWith('```')) {
  raw = raw.replace(/^```(json)?\n|\n```$/g, '');
}
const parsed = JSON.parse(raw);
return [{ json: parsed }];
```

A partir de aquí ya tienes la decisión de Gemini como JSON limpio: decisión, razón, monto y mensaje al cliente.

Paso 10: Code — auditoría y trazabilidad

Registra la decisión completa. TODO: conectar a BD real (Postgres, Elasticsearch).

```
// Trazabilidad y auditoría
const combinado = $('Combinar respuestas').item.json;
const decision = $json;
const audit = {
  request_id: `${$workflow.id}-${Date.now()}`,
  timestamp: new Date().toISOString(),
  customer_id: combinado.cliente.customer_id,
  risk_response: combinado.riesgo, fraud_response: combinado.fraude, crm_response:
combinado.cliente,
  llm_model: 'gemini-2.5-flash', decision: decision.decision, human_review:
decision.human_review,
};
// TODO: insertar 'audit' en tu tabla/índice de trazabilidad real
console.log('AUDITORIA:', JSON.stringify(audit));
return [{ json: decision }];
```

Con esto, cada decisión queda registrada con timestamp, modelo usado y las tres respuestas de origen — lista para auditoría y regulador.

Paso 11: Code — guardrail de validación

Valida la decisión del modelo. Fuerza revisión humana si hay riesgo alto o fraude, aunque Gemini haya dicho lo contrario.

```
const decision = $json;
const fraude = $('Combinar respuestas').item.json.fraude;
// Un monto alto solo fuerza revisión si el riesgo también es ALTO
const montoAltoYRiesgoAlto =
  decision.approved_amount > 10000000 && decision.risk_level === 'ALTO';
// Fraude alto siempre fuerza revisión
const fraudeAlto = fraude.fraud_risk_score >= 45;
const needs_human_review = montoAltoYRiesgoAlto || fraudeAlto;
return [{ json: { ...decision, needs_human_review } }];
```

Paso 12: If — condicional

Campo	Valor
Expression	{{ \$json.needs_human_review }}
Condition	Boolean is true

Paso 13: Set — marcar para revisión (rama TRUE)

Campo	Valor
human_review	true
decision	Pendiente revisión humana
customer_message	Tu solicitud está siendo revisada por nuestro equipo de riesgo. Te contactaremos pronto.

Paso 14: Send Email

Notifica al cliente que su solicitud está en revisión (solo rama TRUE del If).

Campo	Valor
Email to	{{ \$json.customer_email }}
Subject	Solicitud bancaria en revisión

Paso 15: Webhook Response

Retorna la respuesta final al cliente, apruebe o no la solicitud.

Campo	Valor
Respond with	JSON
Response Body	{{ \$json }}

Con esto tu flujo queda completo: de la solicitud del cliente a una decisión trazable, con un humano siempre disponible para el caso sensible.